IE 206 : Introduction to AI & ML

K-Nearest Neighbours and Bayes Methods

Manjesh K. Hanawal

February 6, 2026

K-Nearest Neighbours

Motivation

- When classification algorithms are used in real-world applications, the data is often quite large and difficult to model.
- This limitation warrants a need to have a **simpler** classification algorithm that can classify points with high accuracy and is easy to implement.
- One such algorithm is the **K-Nearest Neighbours** algorithm.

Introduction

- **Simplicity:** K-NN is straightforward and intuitive, making it easier to understand and implement.
- **Non-parametric nature:** Since it is a **nonparametric method**, it does not make assumptions about the underlying data distribution.
- **Effectiveness:** Despite its simplicity, K-NN can achieve high accuracy in many real-world scenarios.
- **Versatility:** K-NN is not restricted to classification, but can be used for regression as well.

Description

- We consider a dataset that has defined classes (binary or multi-class) and each point is classified into one of these classes.
- Then, the model receives an input that needs to be classified into one of these classes.
- The algorithm then find the K points out of the dataset that are *"closest"* to the input point.
- Finally, the algorithm will put the input point into the class that has the maximum number of neighbours and update the dataset.

Algorithm

k-NN

input: a training sample $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$
output: for every point $\mathbf{x} \in \mathcal{X}$,
return the majority label among $\{y_{\pi_i(\mathbf{x})} : i \leq k\}$

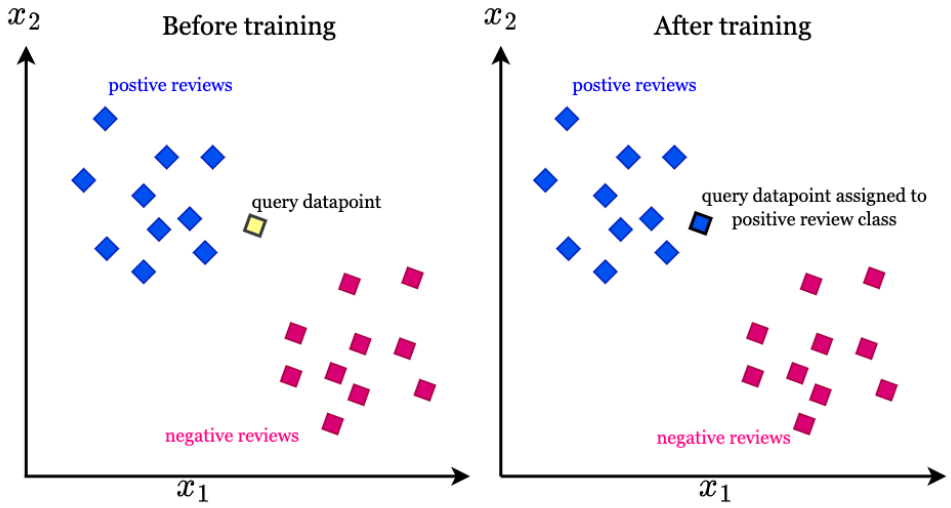

Figure: K-NN Classification

Distance Measures

The algorithm works by measuring the **distance** between the input and other points in the dataset. Given 2 data points $x, y \in \mathbb{R}^n$, we can define multiple distance measures.

- **Eucledian:** $d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
- **Manhattan:** $d(x, y) = \|x - y\|_1 = \sum_{i=1}^{n} |x_i - y_i|$
- **Minkowski:** $d(x, y) = \|x - y\|_p = (\sum_{i=1}^{n} (x_i - y_i)^p)^{\frac{1}{p}}$

Note that when we set $p = 1$ in the Minkowski distance, we end up with the **Manhattan** distance and when we set $p = 2$, we get the **Eucledian** distance.

Limitations

- K-NN is a **lazy learning algorithm** because it uses all the data for training while classification.
- K-NN is affected by the **curse of dimensionality**. When the dimensions go beyond 3, the distance measure makes lesser intuitive sense and becomes difficult to compute.
- As the size of the data increases, the algorithm **slows down significantly** since it must calculate the distance from the input to all the training samples.
- K-NN can be **sensitive to noisy data** because all points contribute to distance calculations.

Bayes Classifier

Introduction

- The Bayes classifier is the classifier having the smallest probability of mis-classification of all classifiers using the same set of features.
- This classifier uses the **Bayes' Thoerem** (hence the name) to find the conditional probability of a **label, given the input** and then chooses the label with the highest conditional probability.
- The Bayes' theorem is shown below -

$$\mathbb{P}(Y|X) = \frac{\mathbb{P}(X|Y)\,\mathbb{P}(Y)}{\mathbb{P}(X)}$$

Features of Bayes Classifier

- **Optimality:** The Bayes classifier is considered optimal because it minimizes the probability of error. It achieves the lowest possible error rate for a given distribution.
- **Error Rate:** The minimal error rate of the Bayes classifier is called the Bayes risk.
- **Assumptions:** The effectiveness depends on the accurate estimation of the probabilities $P(Y)$ and $P(X|Y)$, which can be challenging in practice.
- **Real-World Use:** Despite its theoretical optimality, practical implementations might not achieve the Bayes risk due to limitations in estimating true probabilities from finite data samples.

Optimal Bayes Classifier

- The Optimal Bayes Classifier is the one that shall return the label that maximizes the conditional probability. For the input $x \in \mathcal{X}$ and the space of labels $\mathcal{Y}$, the objective function looks like -

$$h^*(x) \;=\; \arg\max_{y \in \mathcal{Y}} \mathbb{P}(Y = y \mid X = x)$$

- However, this is the case for a multi-class classification. In case of a binary classification task, $\mathcal{Y} \;=\; \{0, 1\}$. This assumption allows for a more simpler expression for the optimal hypothesis.

$$h^*(x) \;=\; \mathbb{I}\{\eta(x) \;>\; 0.5\} \quad ; \quad \eta(x) \;=\; \mathbb{P}(y = 1 \mid x)$$

Why Is It Optimal?

- For any x, Bayes classifier chooses class $h^*(x)$ such that:

$$P(h^*(x) \mid X = x) \geq P(y \mid X = x) \quad \forall\, y$$

- This minimizes:

$$R(h(x) \mid x) = 1 - P(h(x) \mid X = x) \quad \forall\, h$$

- No other decision rule can achieve lower error at x

Bayes Risk

- Bayes risk is the minimum achievable error:

$$R^* = \mathbb{E}_X \left[1 - \max_k P(Y = k \mid X) \right]$$

- Represents an irreducible error due to class overlap

Key Intuition

- Bayes classifier uses full knowledge of data distribution
- Always picks the most likely class
- Any other classifier makes suboptimal decisions for some x
- Therefore, Bayes classifier is **provably optimal**

Proof of Optimality

For every x, $g^*(x)$ minimizes the conditional risk. Therefore,

$$R(h^*) \leq R(g) \quad \text{for any classifier } g$$

Hence, the Bayes classifier is optimal.

Naive Bayes Classifier

Introduction

- **Definition:** Naive Bayes classifiers are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naive) independence assumptions between the features.
- **Base Formula:** $\mathbb{P}(Y|X_1, \ldots, X_n) \approx \mathbb{P}(Y) \prod_{i=1}^{n} \mathbb{P}(X_i|Y)$
- **Feature Independence:** Assumes that each feature X_i contributes independently to the probability $P(Y|X)$, simplifying calculation and analysis.
- **Use Cases:** Popular in spam filtering, sentiment analysis, and document classification due to its simplicity and speed.

Motivation

- **Efficiency:** Extremely efficient in terms of computational resources, which makes it ideal for large datasets.
- **Performance:** Performs surprisingly well in many practical situations, even with the assumption of feature independence.
- **Scalability:** Due to its probabilistic nature and independence assumption, it scales well with an increasing number of predictors and data points.
- **Robustness:** Effective in cases with missing data and can handle an enormous number of features in training data.

Discriminative vs. Generative Methods

- **Discriminative Methods:** Focus on modelling the decision boundary between the classes. They learn the conditional probability $\mathbb{P}(Y|X)$ directly from the data, aiming to find the best classification rule.
- **Generative Methods:** Attempt to model how the data was generated, aiming to understand the joint probability $\mathbb{P}(X, Y)$. They can simulate new data points and are used in both classification and generating new instances.
- **Key Difference:** Discriminative methods learn the boundaries between classes while generative methods model the distribution of individual classes.

Examples of Discriminative and Generative Methods

Discriminative Example

Support Vector Machine (SVM): SVM is a popular discriminative classifier that defines the optimal separating hyperplane maximizing the margin between different classes.

Generative Example

Naive Bayes: A simple generative model that assumes feature independence and estimates the joint probability of the features and labels to predict the class.